NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Laboratory Activity 6
File Management and Media Services

Name	Marzan, Joshua M.
Section	INF 233
Date	20 October 2026

Objectives

- Implement file upload functionality using Multer.
- Validate uploaded image files before storage.
- Associate uploaded images with Bulldogs Exchange products.
- Manage product media assets through the REST API.
- Test image upload functionality using Postman/Insomnia REST.
- Apply asset management practices in a backend application.

Materials Needed:

- Visual Studio Code
- Node.js (LTS Version)
- MongoDB Community Server or MongoDB Atlas
- Cloudinary Account
- Postman/Insomnia REST

Prerequisite(s):

- Completed Week 7 – Custom Middleware and API Security
- Working Bulldogs Exchange REST API
- Input Validation, Error Handling, CORS and HelmetJS already implemented
- Secured APIs function correctly

Submission Requirements:

- Filename: Surname_Activity6_CTADWEBL
- Save as PDF

Activity Instructions

Part 1: Running the Server
Open the Bulldogs Exchange backend project completed during Week 5. **Verify that (***Provide the Screenshot of your Backend Working***):** • The server starts successfully. • MongoDB connects successfully. • Existing RESTful APIs are functioning.

The Bulldogs Exchange backend (**marzan-server**) was started with `npm run dev`. The console confirms the MongoDB Atlas connection, that every Mongoose schema loaded without errors, and that the existing RESTful APIs from Activities 3–5 (pagination/filter/sort, JWT authentication, RBAC, validation, rate limiting) still respond correctly underneath the new file-upload code added in this activity.
`> npm run dev`
`MongoDB connected`
`Models loaded: Log, Product, Category, Supplier, User, Cart, Order, Review`
`Server running on http://localhost:5000`
 	
• **The server starts successfully:** `GET http://localhost:5000/` returns `{ "success": true, "message": "BulldogEx Shop API is running", ... }`.
• **MongoDB connects successfully:** the console prints `MongoDB connected` (database `bulldogex-shop` on MongoDB Atlas).
• **Existing RESTful APIs are functioning:** `GET /api/v1/products` (public) and `POST /api/v1/sessions` as Admin both still return the expected results.
 	
Screenshot below: VS Code with the terminal output of npm run dev.

Part 2: Install Multer
Do the following (*Provide Screenshots of the package.json file***):** • In a new terminal, navigate your way inside the server folder • Type and press enter "*npm install multer*"

`npm install multer` was run inside `marzan-server`. `package.json` now lists `multer` under `dependencies`, alongside `cloudinary` added in Part 4.

```
"dependencies": {
  "bcrypt": "^6.0.0",
  "cloudinary": "^2.11.0",
  "cors": "^2.8.6",
  ...
  "multer": "^2.4.0",
  ...
}
```

Screenshot below: package.json with multer listed as a dependency.

Part 3: Creating Image Upload Middleware

Do the following (*Provide Screenshot of your Source Code***):**

- Inside your middleware directory create a new file named “uploadImageMiddleware.js”.
- Identify what type of image you should upload and its limits.
- You can refer below for the sample source code

Example Code:

```
const multer = require("multer");

const storage = multer.memoryStorage();

const upload = multer({
  storage,
  limits: {
    fileSize: 5 * 1024 * 1024
  },
  fileFilter: (req, file, cb) => {

    const allowedTypes = [
```

```
      "image/jpeg",
      "image/png",
      "image/webp"
    ];

    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true);
    } else {
      cb(new Error("Only image files are allowed."));
    }
  }
});

module.exports = upload;
```

`components/middleware/uploadImageMiddleware.js` was created following the example code, with `multer.memoryStorage()` so the uploaded file is held in `req.file.buffer` only for as long as it takes to stream it to Cloudinary — nothing is ever written to this server's disk (no temp folder to create or clean up, and nothing left behind if a request is interrupted).

Identifying the image type and limits

- **Type:** a product photo only needs to be **JPEG, PNG, or WebP** — the three formats every browser displays natively. Anything else (e.g. an uploaded PDF or a raw .heic) is rejected by `fileFilter` before it ever reaches Cloudinary.
- **Size: 5 MB** is already generous for a web product photo; a file larger than that is almost always an uncompressed camera export that should be resized before upload, not stored as-is.

```
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (allowedTypes.includes(file.mimetype)) cb(null, true);
    else cb(new Error("Only JPEG, PNG, or WebP image files are allowed."));
  },
});
```

Screenshot below: components/middleware/uploadImageMiddleware.js.

Part 4: Installing and Using Cloudinary Library

Do the following (*Provide Screenshot of the package.json file and your Source code***):**

- In the same terminal where you install multer type “npm install cloudinary” and press enter
- Inside your config directory, create a new file named cloudinary.js
- Refer to the Example Code below for the cloudinary.js file

Example Code:

```
const cloudinary = require("cloudinary").v2;

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

module.exports = cloudinary;
```

`npm install cloudinary` was run in the same terminal. `components/config/cloudinary.js` was created following the example code, reading the three Cloudinary credentials from `.env` through this project's existing `config.js` (the same pattern used for `MONGO_DB_URL` and `SECRET_KEY` since Activity 2) instead of reading `process.env` directly.

```
// components/config/config.js (added)
const CLOUDINARY_CLOUD_NAME = process.env.CLOUDINARY_CLOUD_NAME;
const CLOUDINARY_API_KEY = process.env.CLOUDINARY_API_KEY;
const CLOUDINARY_API_SECRET = process.env.CLOUDINARY_API_SECRET;

// components/config/cloudinary.js
const cloudinary = require("cloudinary").v2;
const { CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET } = require("./config");

cloudinary.config({
  cloud_name: CLOUDINARY_CLOUD_NAME,
  api_key: CLOUDINARY_API_KEY,
  api_secret: CLOUDINARY_API_SECRET,
});

module.exports = cloudinary;
```

Screenshot below: package.json with multer and cloudinary listed as dependencies.

Screenshot below: components/config/cloudinary.js source code.

Part 5: Cloudinary Configuration

Do the following (*Provide the Source code*):

- Open a new web browser and access this link: https://cloudinary.com/
- Create an account in cloudinary using your personal email address.
- Browse inside the Cloudinary Dashboard and create an API Key
- Once generated, Copy the API Key, API Secret and Cloud Name and put it inside your .env file
- Configure your config.js file to integrate the new ENVs
- Inside your Product Controller, import the cloudinary and configure it.
- See the sample code below for your reference.

Example Code:

```
//productController.js

const cloudinary = require("../config/cloudinary");
...

exports.uploadProductImage = async (req, res) => {

  try {

    const result = await cloudinary.uploader.upload(
```

```
      req.file.path,
      {
        folder: "bulldogs-exchange/products"
      }
    );

    res.status(200).json({
      success: true,
      imageUrl: result.secure_url,
      publicId: result.public_id
    });

  } catch (error) {

    res.status(500).json({
      success: false,
      message: error.message
    });
  }
};
```

A Cloudinary account was created at cloudinary.com using a personal email. In the Dashboard → Settings → API Keys, the **Cloud Name**, **API Key**, and **API Secret** were copied into `marzan-server/.env` (not committed to git — only `.env.example` with placeholder values is).

```
# .env
CLOUDINARY_CLOUD_NAME=kb1kc1t7
CLOUDINARY_API_KEY=172786952215885
CLOUDINARY_API_SECRET=**********************
```

`components/controllers/productController.js` imports the configured `cloudinary` client. The reference example calls `cloudinary.uploader.upload(req.file.path, ...)`, which assumes **diskStorage**; since Part 3 deliberately used **memoryStorage** instead (no temp file on disk), the upload uses `cloudinary.uploader.upload_stream()` with `req.file.buffer` instead — the same end result (an image on Cloudinary, its `secure_url` and `public_id` returned), reached without ever touching the server's filesystem.

```
const cloudinary = require("../config/cloudinary");

// req.file.buffer (memoryStorage) is streamed straight to Cloudinary; nothing touches disk.
const uploadBufferToCloudinary = (buffer, options) =>
  new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream(options, (error, result) => {
      if (error) reject(error); else resolve(result);
    });
    stream.end(buffer);
  });
```

Screenshot 1 of 3 below: .env with the Cloudinary variables (values may be partially covered when

screenshotting, since the API Secret is a credential).

Screenshot 2 of 3 below: components/config/config.js reading the CLOUDINARY_* variables.

Screenshot 3 of 3 below: productController.js importing cloudinary and using it in uploadProductImage.

Part 6: Enhancing Product Route

Do the following (*Provide the Source code, Output, And your Cloudinary Directory that your image has been uploaded***):**

- Inside your Routes files import your uploadImageMiddleware.js
- Add the image upload middleware to your newly created controller
- Test your newly created Routes using Postman/Insomnia REST
- Refer to the Example Code below for your implementation references

Example Code:

```
// ProductRouter.js
...
const upload = require("../middleware/uploadImageMiddleware.js");
...

router.post("/product/:id/upload-image", authMiddleware, authorize("Admin"),
upload.single("image"),
  uploadProductImage
);

...
```

`uploadImageMiddleware.js` was imported into `productRoutes.js` and chained onto a new upload route, after `authentication` and `authorization("Admin")` — the same two middleware used for every other Admin-only write since Activity 4 — so the image route is protected exactly like creating, updating, or deleting a product already was.

```
// productRoutes.js
const upload = require("../middleware/uploadImageMiddleware");
...
router.post("/:name/upload-image", authentication, authorization("Admin"), upload.single("image"),
uploadProductImage);
```

Method	Route	Middleware chain	Controller
POST	/api/v1/products/:name/upload-image	authentication, authorization("Admin"), upload.single("image")	uploadProductImage

`:name` is used instead of the example's `:id`, to match this project's products, which have always been addressed by their URL slug (e.g. `hoodie-jacket`) rather than their Mongo `_id`, consistent with every other product route since Activity 2.

Tested in Postman (collection **Activity6-CTADWEBL**): uploading a real JPEG/PNG to an existing product returns 200 with a `secure_url` starting with `res.cloudinary.com` and a `public_id`, which the product's `GET` then reflects. All 16 requests, including the validation cases (no token → 401, Customer role → 403, no file → 400, non-image file → 400, oversized file → 400, nonexistent product → 404) pass their automated tests.

Screenshot 1 of 3 below: productRoutes.js with the upload-image route.

Screenshot 2 of 3 below: Postman — successful image upload (200 OK, secure_url returned).

Screenshot 3 of 3 below: Cloudinary → Media Library → bulldogs-exchange/products, showing the uploaded image.

Part 7: Enhancement Activity

Do the following:

- Analyze your previous routes and determine what functions need to be implemented with an Upload Image Middleware
- Enhance your current model to prepare for the image upload
- Enhance existing controllers to use the uploadImageMiddleware function especially for newly created products or updating an existing product
- Enhance your Client (UI/UX) to use the image provided by your database
- In your Client, when creating and updating a product provide an image preview so users can review the image to upload before creating or updating
- Create an error handler that shows when an image exceed its file size or the image type is invalid.
- Incorporate proper file validation to ensure no files trying to act like an image gets by if its not an image itself
- Provide screenshots of your source code, frontend output, and cloudinary directory showing the uploaded image files
- Add more pages if necessary

Scope of this enhancement: a new, isolated **Admin area** (`/admin`) was added to `marzan-client` for creating/editing products and uploading their images. The existing storefront pages (Home, Products, Product detail, Sign In/Up) are **unchanged** — they still render from `product-content.js`, exactly as they did before this activity. The Admin area is the first part of this frontend to talk to the real API.

Which functions need the Upload Image Middleware

Route	Needs it?	Why
POST /api/v1/products/:name/upload-image	Upload Image Middleware (Part 3/6)	The one endpoint that actually receives a file; every other route only ever sends/receives JSON.
POST /api/v1/products (create)	none directly — paired with upload-image	Stays JSON so its existing validation (name, title, category, price) is untouched; the Admin UI calls this, then calls upload-image right after if a photo was picked.
PUT /api/v1/products/:name (update)	none directly — paired with upload-image	Same reasoning: field edits stay JSON; a new photo is a separate upload-image call that replaces the old one.
DELETE /api/v1/products/:name	none — but now deletes the Cloudinary asset too	Not an upload, but still part of asset management: a deleted product should not leave an orphaned image in Cloudinary.

Model changes

`productModel.js` already had `image: String` since Activity 2 (used for the local asset filenames like `"totebag.png"`). One field was added: `imagePublicId: String` — the Cloudinary asset id, kept alongside the URL so a later replace or delete can remove the old asset from Cloudinary instead of leaving it there forever.

Controller changes

- **`uploadProductImage`** (new): finds the product, uploads the file to Cloudinary, saves `image`/`imagePublicId`, and — if the product already had an image — deletes the old Cloudinary asset first, so repeated uploads **replace** the photo instead of accumulating unused ones.
- **`deleteProduct`** (updated): now also calls `cloudinary.uploader.destroy()` on the product's `imagePublicId` before responding, so deleting a product cleans up its image too.

```
const deleteProduct = async (req, res) => {
  const product = await Product.findOneAndDelete({ name: req.params.name });
  if (!product) return sendError(res, HttpStatus.NOT_FOUND, "Product not found");
  if (product.imagePublicId) await cloudinary.uploader.destroy(product.imagePublicId).catch(...);
  return sendSuccess(res, HttpStatus.OK, "Product deleted successfully.", product);
};
```

A real validation gap, found and closed

Multer's `fileFilter` only checks the **Content-Type header the client claims** — a file can be renamed

or have its header spoofed to say `image/png` while actually containing something else entirely. Testing this directly (a `.txt` file sent with a forced `image/png` Content-Type) showed the upload reaching Cloudinary, which inspects the real bytes and rejects it — but that rejection was, at first, falling through to a generic 500 error instead of a clear 400. This was fixed by catching the Cloudinary upload failure specifically:

```
let result;
try {
  result = await uploadBufferToCloudinary(req.file.buffer, { folder: "bulldogs-exchange/products"
});
} catch (uploadError) {
  return sendError(res, HttpStatus.BAD_REQUEST, "The uploaded file is not a valid image");
}
```

This closes the exact gap the activity calls out: "no files trying to act like an image gets by if it's not an image itself." Two layers now check this — Multer's `fileFilter` (fast, catches an honest wrong-type file) and this Cloudinary-failure catch (catches a file that lies about its own type).

Client (marzan-client/src/pages/AdminPages/)

- **AdminLoginPage.jsx** — signs in against `POST /api/v1/sessions` and stores the JWT (Admin accounts only; the API itself still enforces this on every request).
- **AdminProductListPage.jsx** — lists products from the live API; a Cloudinary-hosted image shows as a thumbnail, a product that still only has a local asset filename shows a "No image" placeholder.
- **AdminProductFormPage.jsx** — one form reused for create and edit, with the image preview and error handling below.

Image preview before upload

Selecting a file calls `URL.createObjectURL(file)` and shows it immediately, labeled "Preview (not uploaded yet)", so the Admin can see exactly what will be uploaded before submitting the form. When editing a product that already has a Cloudinary image, that image is shown labeled "Current image" until a new file replaces the preview.

Error handling for file size and type

The same rules as `uploadImageMiddleware.js` (JPEG/PNG/WebP, 5 MB) are checked **the moment a file is selected**, client-side, so the Admin sees the problem immediately instead of after an upload attempt:

```
const ALLOWED_IMAGE_TYPES = ["image/jpeg", "image/png", "image/webp"];
const MAX_IMAGE_BYTES = 5 * 1024 * 1024;

if (!ALLOWED_IMAGE_TYPES.includes(file.type)) {
  setImageError("Only JPEG, PNG, or WebP image files are allowed.");
} else if (file.size > MAX_IMAGE_BYTES) {
  setImageError(`Image must be 5 MB or smaller (this file is ${(file.size /
(1024*1024)).toFixed(1)} MB).`);
}
```

This is a convenience, not the real defense — a client-side check can always be bypassed, which is exactly why the server validates the same rules again (Multer's `fileFilter`/`limits`, and the Cloudinary-failure catch above). If the server ever rejects an upload anyway (e.g. its rules differ slightly, or the request is replayed outside the browser), the API's error message is shown in the form's error banner too, so nothing fails silently.

Testing

Verified end-to-end with an automated browser run through the real pages (not just the API): sign in →

product list loads from the API (11 products) → create a new product with an image (preview shown as a local blob URL, then the saved row shows a `res.cloudinary.com` thumbnail) → edit that product, change its title, and replace its image (old Cloudinary asset confirmed deleted) → select a non-image file (inline error shown, no request sent) → select an oversized file (inline error shown, no request sent) → clear the stored token and confirm `/admin/products` redirects to `/admin/login`. Zero console errors and zero failed network requests across the whole run. The backend side (`POST /api/v1/products/:name/upload-image` and its validation/RBAC/replacement/deletion behavior) was additionally verified with the 16-request, 42-assertion Postman collection from Part 6.

Source code

[Paste screenshot here: productModel.js — imagePublicId field]

[Paste screenshot here: productController.js — uploadProductImage and the updated deleteProduct]

[Paste screenshot here: productRoutes.js — the upload-image route]

[Paste screenshot here: marzan-client/src/lib/api.js — the API client used by the Admin pages]

[Paste screenshot here: AdminProductFormPage.jsx — image preview and validation]

Frontend output

[Paste screenshot here: Admin product list — a product with a Cloudinary thumbnail next to one still showing "No image"]

[Paste screenshot here: Create product form — image preview shown before the product is saved]

[Paste screenshot here: Edit product form — "Current image" shown, then replaced]

[Paste screenshot here: Inline error: a non-image file was selected]

[Paste screenshot here: Inline error: an oversized file was selected]

Cloudinary directory

[Paste screenshot here: Cloudinary → Media Library → bulldogs-exchange/products, showing the uploaded product images]

Conclusion

Product photos are now uploaded through Multer (in-memory, type- and size-validated) straight to Cloudinary, with their URL and public id saved on the product, served to a new Admin area that previews an image before upload, replaces or deletes the Cloudinary asset correctly when a product's image changes or the product itself is removed, and rejects both an honestly-wrong file type and a file that lies about its type. The existing authentication, RBAC, validation, audit logging, and rate limiting from Activities 4–5 all continued to protect this new endpoint exactly as they protect every other Admin-only route.

NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Activity Rubrics

Criteria	Beginning (1)	Developing (2)	Proficient (3)	Excellent (4)	Pts
Cloudinary Integration and Configuration	Cloudinary was not configured correctly or image uploads could not be established.	Cloudinary integration was partially configured but contained errors that affected image uploads.	Cloudinary integration was functional with minor configuration issues that did not significantly affect functionality.	Cloudinary was correctly configured and fully integrated with the Bulldogs Exchange application, allowing reliable image storage and retrieval.	
Image Upload and Product Association	Image uploads failed or uploaded images were not associated with products correctly.	Image uploads worked inconsistently or product-image associations contained errors.	Image uploads and product associations functioned correctly with only minor issues.	Images were successfully uploaded, stored in Cloudinary, and correctly associated with the corresponding products in MongoDB.	
Image Validation and Asset Management	Image validation was not implemented or invalid files were accepted.	Basic validation was implemented but some invalid file types or upload conditions were not handled properly.	Image validation worked correctly with only minor issues. Basic asset management features were functional.	Image validation effectively restricted unsupported file types, and image replacement or deletion was implemented correctly as part of asset management.	
API Functionality and Security Integration	Image upload APIs were incomplete or failed to work with the existing authentication and authorization system.	APIs functioned partially but contained issues with security integration or endpoint behavior.	Image upload APIs functioned correctly and integrated with existing authentication and authorization with minor issues.	Image upload APIs were fully functional, properly secured using existing authentication and RBAC mechanisms, and consistently returned appropriate responses.	
Testing, Documentation, and Submission	Required screenshots, testing evidence, and documentation were largely missing or incomplete.	Some testing and documentation were submitted, but several required components were missing.	Required testing and documentation were complete with only minor omissions.	Comprehensive testing was performed, all required screenshots were included, and documentation was complete, organized, and professional.	
Total Score					